딥러닝 프로그래밍 실습 #4

전력망 안정성, 회귀로 풀까 분류로 풀까

MSE vs 교차 엔트로피 비교, 그리고 fit() 12회 예산 안에서 옵티마이저·학습률 고르기

UCI Electrical Grid Stability · 10,000건 · 4노드 별형 전력망

2022204065 윤성근

목표 0 · 실행 전 가설

코드를 돌리기 전에 적어 둔 예측

가설 1 · 회귀 vs 분류

stab 회귀 후 0 기준 판정이 직접 분류보다 정확도가 높을 것

이유: 전체 평균으로 봤을 때 더 좋은 점수가 나올 것 같다. 연속값이 '얼마나 불안정한지'까지 담고 있어 정보가 더 많다.

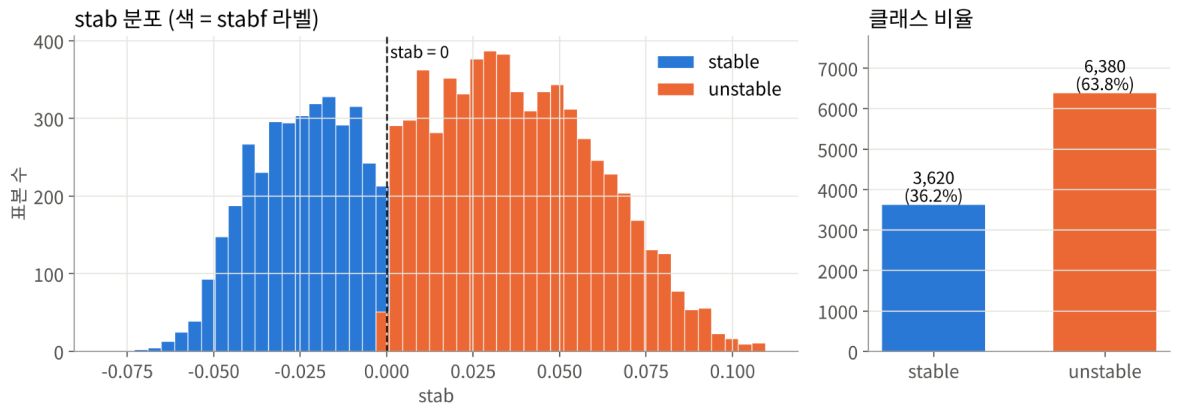
가설 2 · 옵티마이저

SGD, 모멘텀, Adam, RMSprop 중 Adam이 가장 좋을 것

이유: 보통 기본값으로 많이 쓰는 옵티마이저라 무난할 것 같다.

목표 1 · 데이터 탐색, 신뢰성 검증, 전처리

데이터는 믿을 만했고, stab과 p1은 뺐다



stab 제외 = 정답 누설 차단. stab > 0이면 무조건 unstable이라, 넣으면 정확도 100% 짜리 쓸모없는 모델이 된다.

p1 제외 = 중복 정보. 생산량 p1은 소비량 p2 + p3 + p4의 합과 정확히 같아 새 정보가 없다 → 입력 11개.

분할·정규화. train/test 8:2, stratify, seed 42. StandardScaler는 train에만 fit.

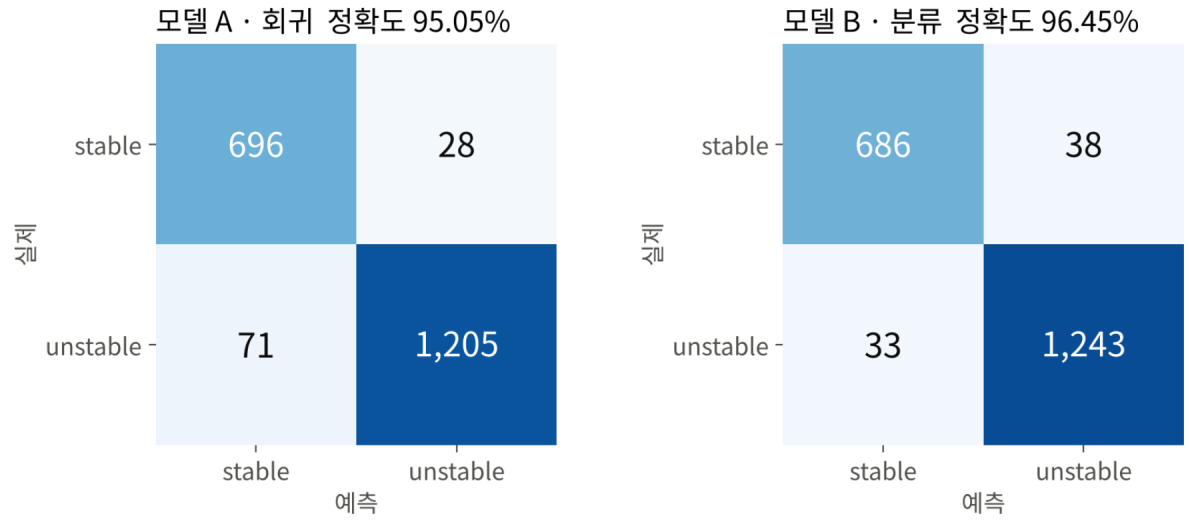
신뢰성 검증 8가지 — 모두 통과

- **결측치** 0건, 라벨은 2종뿐
- **중복** 같은 행·순서만 바꾼 복사본 0건
- **값 범위** tau·g·p 모두 설명서 범위 안
- **물리 법칙** 생산 = 소비 합 (오차 6×10^{-15})
- **라벨 일관성** stab > 0 ↔ unstable 100%
- **이상치** 표준화 후 |z| > 3인 값 0개
- **분할** 비율 63.8% 동일, KS p > 0.05, 겹침 0
- **입력 상관** p1 제외 후 최대 0.02

⚠ **한계** 시뮬레이션 데이터라 실제 측정 잡음이 없다. 현실에선 정확도가 더 낮을 수 있다.

목표 2 · 회귀 vs 분류

같은 조건에서 분류가 1.4%p 더 정확했다



test 2,000건	A 회귀 (MSE)	B 분류 (BCE)
정확도 (seed 42)	95.05%	96.45%
3개 seed 평균	95.62%	96.65%
오판정 수	99	71

공정 비교를 위해 고정한 조건

- 은닉층 64 → 32 (ReLU), 2개
- Adam, lr 0.001, batch 32, 50 epoch
- train 중 20%를 검증용으로 분리
- 가중치 초기화 seed 동일

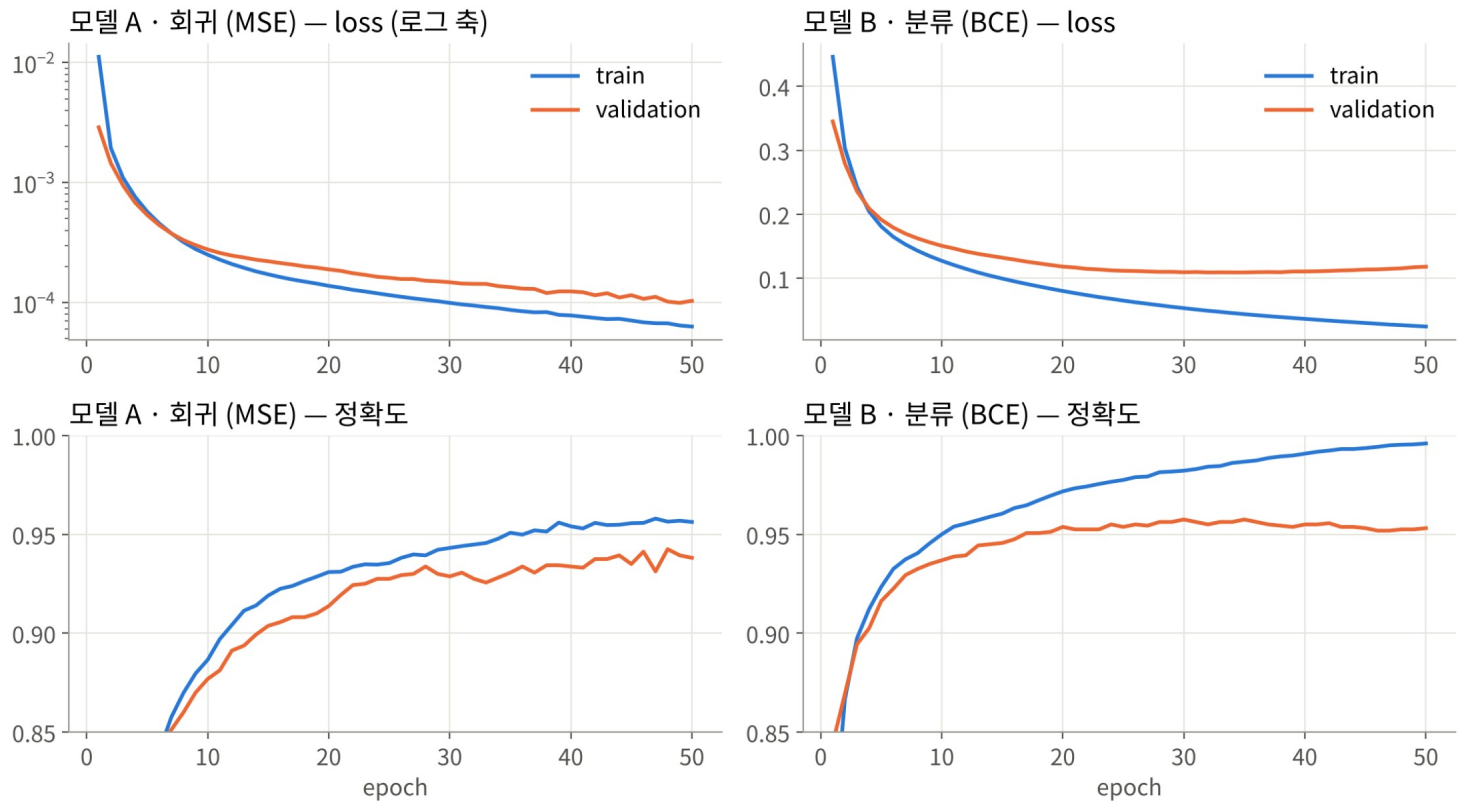
다른 것은 출력·손실뿐

A: linear 1개 + MSE, 예측 > 0이면 unstable
B: sigmoid 1개 + 이진 교차 엔트로피

seed 1에서는 96.30 vs 96.25로 거의 같았다. 그래서 한 번의 실행만 믿지 않고 seed 3개로 확인했다.

목표 2 · 학습 곡선

두 모델 모두 정상 학습, 수렴 속도는 달랐다



B 분류: 30 epoch 이후 과적합 조짐

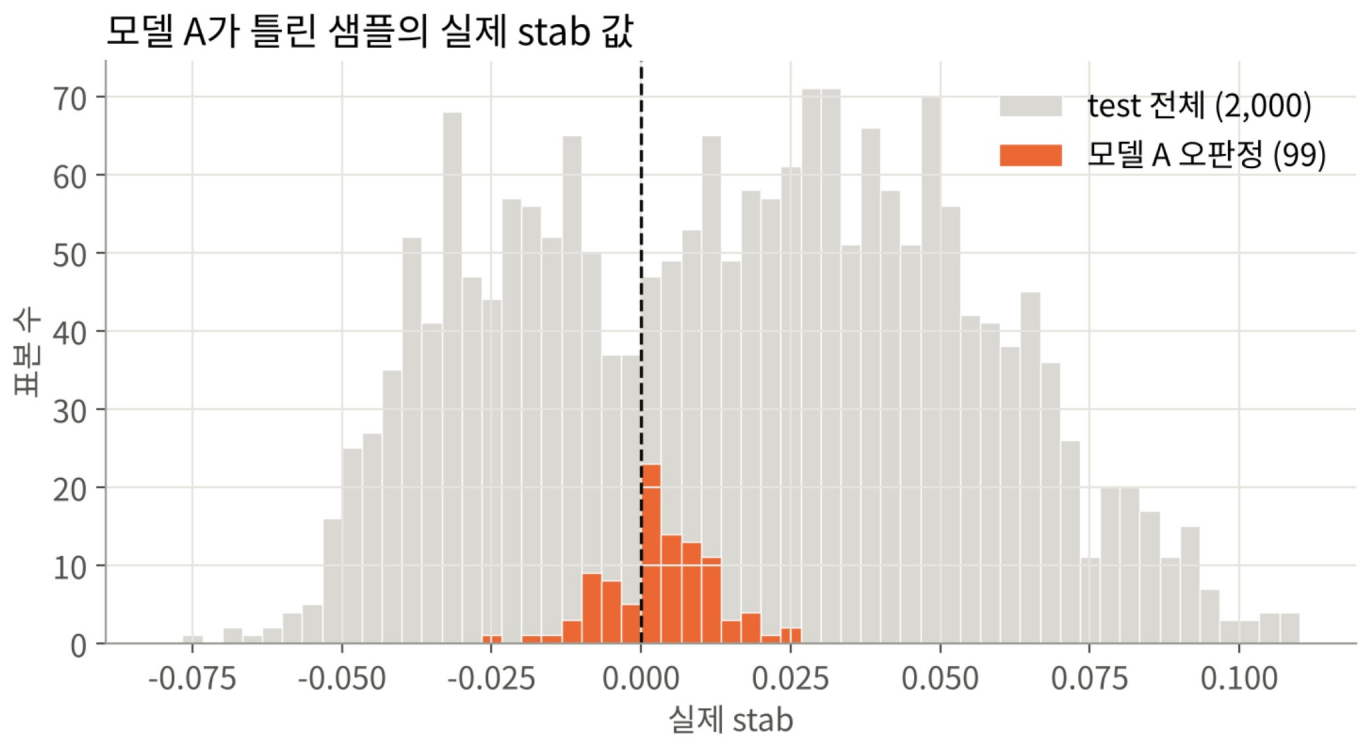
validation loss가 ~30 epoch에서 바닥을 찍고 다시 오른다. train 정확도는 99.6%까지 가지만 validation은 95%대에서 멈춘다.

A 회귀: 50 epoch에도 아직 내려가는 중

정확도가 90%를 넘기까지 10 epoch 이상 걸렸고 끝까지 오르는 중이다. 같은 epoch 수가 회귀에 약간 불리했을 수 있다.

목표 2 · 오판정 분석

회귀의 실수는 stab = 0 바로 옆에 몰려 있다



73%

오판정 중 |stab| < 0.01
(test 전체는 13.7%)

0.006

오판정의 |stab| 중앙값
(전체 0.031의 1/5)

의미와 결론

MSE는 값이 얼마나 가까운지만 줄이고 부호는 따로 신경 쓰지 않는다. 게다가 예측이 평균 0.0025 낮게 치우쳐 불안정을 안정으로 71건 놓쳤다. 기준을 −0.003으로 옮겨 보정해도 95.65%로 분류(96.45%)에 못 미친다 → **분류를 택한다.**

목표 3 · 탐색 계획 (분류 모델)

5회는 넓게, 6회는 깊게, 1회는 최종 학습에

1단계 · 5회 · hold-out

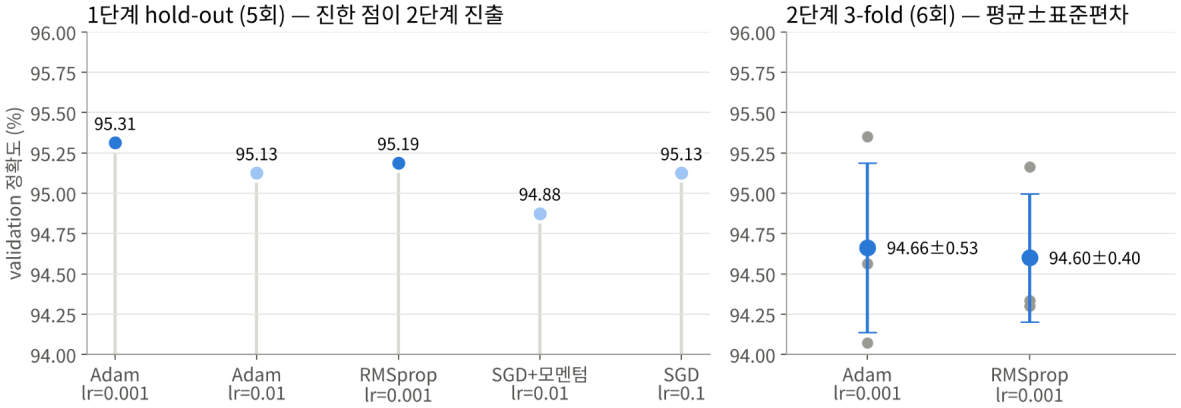
옵티마이저 4종을 모두 한 번씩, 각자 쓸 만한 학습률로. Adam만 0.001/0.01 두 개. SGD는 0.1, 모멘텀은 0.01(관성 0.9로 유효 보폭 ≈ ×10).

2단계 · 6회 · 3-fold CV

1단계 상위 2개만 3-fold로 다시 재서 평균과 편차를 본다.

최종 · 1회

고른 조합으로 train 8,000건 전체 재학습 → test는 딱 한 번 평가.



Trade-off 판단

4×3 hold-out 12회는 넓지만 한 번씩만 재서 운에 휘둘리고, 4조합×3-fold는 믿을 만하지만 좁다. 그래서 둘을 나눴다. 실제로 Adam은 hold-out 95.31% → CV 평균 94.66%로 0.65%p 낮아졌는데, 이는 1단계 조합 간 최대 차이(0.43%p)보다 컸다. 넓게만 봤다면 잡음으로 순위를 정했을 것이다.

목표 3 · 실행 로그와 최종 결과

Adam · lr 0.001, test 정확도 97.05%

#	단계	옵티마이저	lr	val 정확도	val loss	초
1	hold-out	Adam	0.001	95.31%	0.124	23.9
2	hold-out	Adam	0.01	95.13%	0.245	23.5
3	hold-out	RMSprop	0.001	95.19%	0.132	22.9
4	hold-out	SGD+모멘텀	0.01	94.88%	0.180	21.5
5	hold-out	SGD	0.1	95.13%	0.178	20.9
6	CV fold 1	Adam	0.001	95.35%	0.131	23.7
7	CV fold 2	Adam	0.001	94.56%	0.155	25.5
8	CV fold 3	Adam	0.001	94.07%	0.149	25.3
9	CV fold 1	RMSprop	0.001	95.16%	0.144	22.4
10	CV fold 2	RMSprop	0.001	94.30%	0.159	22.0
11	CV fold 3	RMSprop	0.001	94.34%	0.157	23.7
12	최종 재학습	Adam	0.001	test 97.05%	–	21.5

CV 평균 ± 표준편차

Adam 94.66 ± 0.53%
RMSprop 94.60 ± 0.40%
차이 0.06%p ≪ 편차 → 사실상 동률

최종 test (2,000건)

오판정 59건 (stable→unstable 16, 반대 43). 목표 2의 96.45%보다 높은 이유: 학습 데이터 6,400 → 8,000건

계산 비용

1회 평균 23.1초 (2코어 CPU), 12회 4.6분
전수 탐색 4종×5 lr×5-fold = 100회 ≈ 38분

회고 · 예측과 달랐던 점

가설 1은 틀렸고, 가설 2는 맞았지만 차이가 없었다

가설 1 · 회귀가 더 정확할 것

✗ 틀림

분류 96.45% vs 회귀 95.05%
(seed 3개 평균 96.65 vs 95.62)

‘전체 평균으로 더 좋을 것’이 함정이었다. MSE는 평균 오차를 줄이는 데 힘을 쓰지만, 정확도는 0 근처의 부호로만 갈린다. 회귀 오판정의 73%가 바로 그 구간에서 나왔다.

가설 2 · Adam이 가장 좋을 것

△ 맞았지만 차이 미미

Adam 94.66% vs RMSprop 94.60%
(CV, 편차 ±0.4~0.5%p)

1위는 맞았지만 1단계 5개 조합이 모두 0.43%p 안이었다. 옵티마이저보다 학습률이 중요했다. 같은 Adam도 lr 0.01에서는 val loss가 두 배(0.124 → 0.245)였다.

⚠ **더 해본 것 · 정확도가 전부는 아니다.** 분류 판정 기준을 0.5 → 0.3으로 낮추면 위험한 오류(불안정을 안정으로 판정)가 33 → 21건(−36%)으로 줄고, 정확도는 96.45 → 96.25%로 0.2%p만 손해. 정전 위험을 생각하면 기준을 낮추는 쪽이 낫다.

AI 도구 활용 내역

Claude로 분석했고, 틀린 그래프 하나를 잡아냈다

사용 도구와 핵심 프롬프트

Claude (claude.ai) — 코드 실행(Keras), 그래프, 슬라이드 초안

① 과제 PDF + CSV 첨부 → “코드 실행 + 슬라이드까지” 요청

② “회귀 vs 분류가 무슨 말인지 모르겠어” → 설명을 듣고 가설은 실행 전에 내가 직접 작성

③ “내용이 어려우니 쉽게 설명해줄래?” → 전체 흐름을 이해한 뒤 발표 준비

④ “데이터 신뢰성 검증 페이지도 만들었으면 좋겠어” → 8가지 검증 추가, 10쪽 제한에 맞게 “3쪽과 4쪽을 합쳐줘”

⑤ “남들과 차별성을 줄 방법은?” → 제안 중 판정 기준 분석을 골라 6·9쪽에 반영

AI의 출력이 틀렸던 사례

회귀 모델 정확도가 55%로 나온 학습 곡선

증상 회귀 정확도 곡선이 50 epoch 내내 52~55%였다. 그런데 test 정확도는 95%라 앞뒤가 맞지 않았다.

원인 AI가 만든 부호 일치 지표에서 정답 (n,)과 예측 (n,1)의 모양이 달라 n×n으로 브로드캐스팅돼, 엉뚱한 쌍끼리 비교하고 있었다.

수정 두 텐서를 1차원으로 reshape → 학습 곡선 정확도 95.6%로 정상화. 모델 자체(손실)는 영향이 없어 test 결과는 그대로였다.